Birkhoff Decompositions and Photonic Interconnects Wait! Don't Forget the Compute!

Eliezer Amponsah
Purdue University
USA

Vamsi Addanki
Purdue University
USA

Abstract

The growing demand for efficient communication in distributed training and inference has sparked significant interest in reconfigurable photonic interconnects across both academia and industry. Mixture-of-Experts (MoE) models, with their highly skewed communication patterns, present a natural opportunity for such circuit-switched fabrics. However, existing approaches largely optimize communication in isolation, overlooking the interaction between communication and the expert computation that follows.

In this paper, we revisit circuit scheduling for all-to-all communication in MoE execution. We show that the dispatch–compute–combine structure fundamentally challenges classical scheduling techniques such as Birkhoff–von Neumann (BvN) decomposition. First, MoE communication matrices are rarely doubly stochastic, introducing significant scheduling bubbles in BvN-based schedules. Second, while decomposition enables communication–compute overlap, the excessive number of matchings produced by BvN fragments execution into small batches, leading to severe compute inefficiencies due to fixed execution overheads. Motivated by these observations, we explore a simple greedy max-weight decomposition strategy that bounds the number of matchings while preserving large batch sizes per matching. Despite its simplicity, the approach significantly improves overlap efficiency, reduces compute overheads, and approaches the performance of an ideal congestion-free all-to-all.

CCS Concepts

• **Networks** → **Data path algorithms**; *Data center networks.*

Keywords

Mixture of Experts, Photonic Interconnect

ACM Reference Format:
Eliezer Amponsah and Vamsi Addanki. 2026. Birkhoff Decompositions and Photonic Interconnects Wait! Don't Forget the Compute! . In *ACM SIGCOMM Workshop on Hot Topics in Optical Technologies and Applications in Networking (HotOptics '26), August 17–21, 2026, Denver, CO, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3789240.3822610

1 Introduction

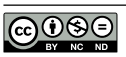
Optical circuit switching has emerged as a promising technology for next-generation datacenter networks [2, 4, 6, 7, 12, 21, 24–27, 31, 36]. Compared to traditional electronic packet switching, optical circuits offer substantially higher bandwidth and lower latency, making them particularly attractive for communication-intensive workloads such as large-scale distributed training and inference [1, 19, 21, 36]. However, these benefits come with an important constraint: a circuit-switched network can only realize a limited communication pattern at a given time and must repeatedly reconfigure itself to support changing traffic demands. Consequently, a significant body of work has focused on topology reconfiguration and circuit scheduling algorithms for efficiently mapping dynamic communication workloads onto optical fabrics [5, 9, 16, 20–23, 28, 29, 33].

At the same time, modern AI workloads are becoming increasingly communication dominated [22]. In particular, Mixture-of-Experts (MoE) architectures introduce highly irregular all-to-all communication due to dynamic token routing across distributed experts [14, 35]. Unlike traditional collective communication patterns, MoE traffic matrices are often sparse and skewed, varying significantly across iterations and inputs. This makes MoE workloads a natural fit for reconfigurable optical fabrics capable of adapting the topology to the communication demand.

A common approach in circuit scheduling is to decompose a traffic matrix into a sequence of matchings that can be executed over time, often using techniques such as Birkhoff–von Neumann (BvN) decomposition [8, 32]. At first glance, such decompositions appear attractive for distributed MoE execution: each matching directly translates into a communication schedule between GPUs, naturally limiting contention while enabling the network to adapt to the routing pattern of the current batch. Furthermore, decomposition implicitly partitions the all-to-all into smaller communication phases, creating opportunities to overlap communication with expert computation.

However, our study reveals that decomposition quality for communication is fundamentally different from decomposition quality for end-to-end MoE execution. In particular, the interaction between communication scheduling and downstream expert computation plays a central role in determining overall performance. While fine-grained decompositions improve network utilization and reduce contention, they also fragment execution into increasingly smaller token batches. In practice, expert execution exhibits substantial fixed overheads at small batch sizes, causing compute efficiency to collapse despite improved communication structure. As a result, decomposition strategies that appear favorable from a purely networking perspective can significantly degrade end-to-end makespan.

This tension is particularly pronounced for the widely studied Birkhoff–von Neumann (BvN) decomposition. Because MoE communication matrices are rarely doubly stochastic, BvN often produces decompositions with a large number of matchings and substantial idle capacity. The resulting execution suffers not only from additional reconfiguration overheads, but also from poor compute efficiency due to excessively fragmented expert batches.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
SIGCOMM'26, Denver, CO, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2467-1/26/08
https://doi.org/10.1145/3789240.3822610

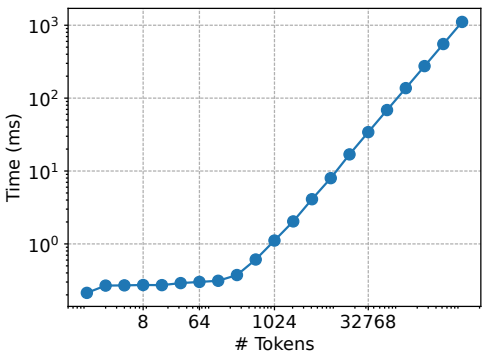


Figure 1: MoE expert compute time across token batch sizes typically exhibits a "knee" behavior. While execution remains approximately linear beyond 256 tokens, smaller batches incur substantial fixed overheads, causing arbitrary-size BvN decompositions to significantly underperform.

Our results show that, in many settings, minimizing communication congestion alone is insufficient and can even be counterproductive.

Motivated by these observations, we instead view distributed MoE execution as a scheduling problem that jointly considers communication decomposition and compute efficiency. Rather than pursuing highly fragmented decompositions, our goal is to preserve large execution batches while still exposing opportunities for communication–compute overlap. We show that a simple max-weight decomposition strategy is remarkably effective in this setting, substantially reducing execution makespan. In contrast, highly fragmented decompositions such as BvN suffer significant performance degradation due to poor compute efficiency at small batch sizes. By preserving larger execution batches, the max-weight approach exposes additional overlap opportunities between communication and expert computation, allowing it in many settings to approach and even outperform an idealized congestion-free all-to-all execution.

Motivated by these observations, our goal is to preserve large execution batches while maintaining communication–compute overlap. We show that a max-weight decomposition strategy is remarkably effective in reducing execution makespan. In contrast, BvN suffers significant performance due to poor compute efficiency at small batch sizes. The max-weight approach exposes additional overlap opportunities between communication and expert compute, allowing it to approach and even outperform an idealized congestion-free all-to-all execution.

We validate these observations using trace-driven simulations across various MoE workloads and decomposition strategies. Our results demonstrate that decomposition granularity, rather than communication efficiency alone, is a first-order concern in distributed MoE execution over reconfigurable optical fabrics.

2 Motivation

We present a brief background and motivate the need for rethinking circuit scheduling techniques in the context of distributed MoE execution.

2.1 Mixture of Experts (MoE) Execution

Mixture-of-Experts (MoE) architectures replace dense feed-forward layers with a pool of specialized "experts," where a learned routing gate dynamically selects only a sparse subset of experts per token [13, 30]. Modern MoE models employ increasingly large expert pools; for example, Qwen3 contains 128 experts with top-8 routing [34], while Mixtral 8×7B employs 8 experts with top-2 routing [17]. This sparsity naturally enables expert parallelism (EP), where experts are distributed across multiple GPUs or machines.

Consequently, MoE execution introduces a tightly coupled dispatch–compute–combine structure:

- **Dispatch:** routed tokens are exchanged across devices through all-to-all (A2A) communication to reach their assigned experts.
- **Compute:** each expert processes its assigned tokens.
- **Combine:** processed tokens are returned to their originating devices through another A2A communication.

The backward pass mirrors this structure, resulting in four all-to-all communications per MoE layer during training. Unlike dense Transformer workloads dominated by collective operations such as AllReduce, MoE execution introduces substantial all-to-all communication together with strict sequential dependencies between communication and expert computation. As a result, the efficiency of MoE execution depends not only on communication performance, but also on how effectively communication and expert computation can be overlapped.

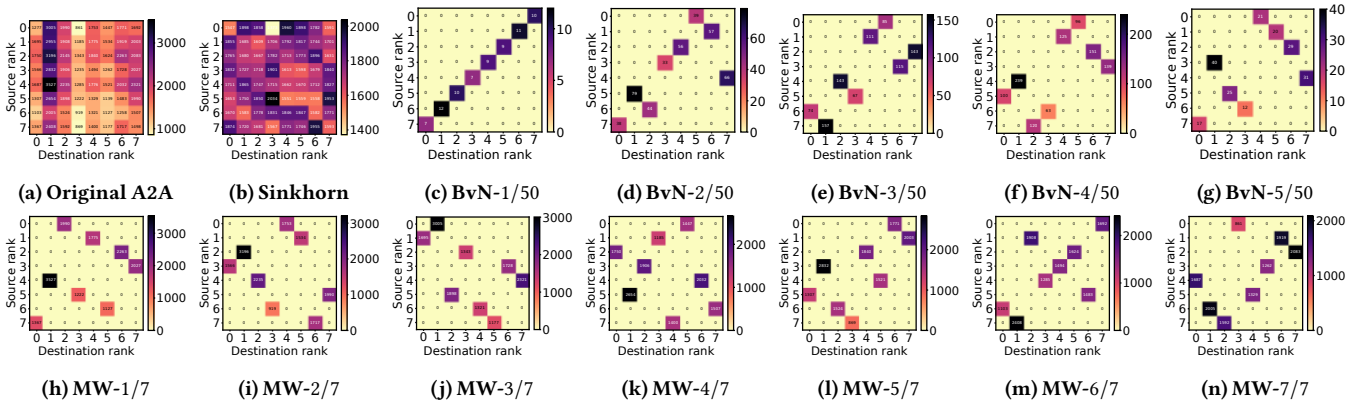
2.2 Communication & Compute Interaction

Interestingly, the communication matrices generated by MoE execution possess many of the properties that motivate reconfigurable circuit scheduling. Because expert assignments are determined dynamically by the routing gate, MoE traffic matrices are often sparse, highly skewed, and imbalanced, varying significantly across iterations and inputs. As a result, distributed MoE execution becomes a natural target for reconfigurable optical interconnects.

The decomposition techniques already employed in OCS scheduling can therefore be directly applied to MoE communication matrices, transforming the all-to-all communication into a sequence of scheduled matchings. At first glance, this appears highly attractive: decomposition not only adapts the network to the skewed communication structure of MoE workloads, but also creates opportunities to pipeline communication, expert computation, and network reconfiguration overheads[16].

However, decomposition does more than define the communication schedule of the network. In MoE execution, each matching also implicitly determines the granularity at which expert computation is exposed. Consider a decomposition that partitions a large all-to-all exchange into many small matchings. While such a schedule may improve instantaneous network utilization and reduce contention, each matching now carries only a small number of routed tokens per expert. Since experts may begin computation immediately upon receiving tokens, the resulting expert executions become increasingly short and fragmented.

Compute time is not strictly linear: Expert execution across token batch sizes typically exhibits a "knee" behavior (Figure 1). Beyond a sufficiently large batch size, execution scales approximately linearly



(a) Original A2A (b) Sinkhorn (c) BvN-1/50 (d) BvN-2/50 (e) BvN-3/50 (f) BvN-4/50 (g) BvN-5/50

(h) MW-1/7 (i) MW-2/7 (j) MW-3/7 (k) MW-4/7 (l) MW-5/7 (m) MW-6/7 (n) MW-7/7

Figure 2: BvN decomposition results in a large number of matchings with small token counts, causing significant compute overheads for MoE execution. Whereas, the max-weight decomposition (MW) bounds the number of matchings to $O(n)$ and maintains large token counts per-matching. Note the colorbar scale.

with the number of tokens. However, at small batch sizes, fixed kernel launch, synchronization, and scheduling overheads dominate execution time, causing compute efficiency to collapse. Kernel level optimisations remain an active area of research [3, 10]. Nevertheless, highly fragmented decompositions may produce communication-efficient schedules that are computationally inefficient.

Communication and compute create competing objectives: Fine-grained decompositions improve network utilization, reduce contention, and expose additional opportunities for communication–compute overlap. At the same time, they shorten expert execution windows and reduce the amount of computation available to hide subsequent communication and reconfiguration overheads. When expert computation becomes too short, the pipeline between communication and compute breaks down, creating scheduling bubbles where both network and compute resources remain underutilized.

Consequently, decomposition quality for communication is fundamentally different from decomposition quality for MoE execution. Effective schedules must jointly balance communication efficiency, overlap opportunities, and execution granularity rather than optimizing network utilization alone.

2.3 Scheduling Challenge in MoE Execution

These observations fundamentally reshape the role of decomposition in distributed MoE execution. Traditionally, decomposition is viewed purely as a communication primitive whose objective is to improve network utilization and reduce contention. However, in MoE execution, decomposition simultaneously determines the ordering, granularity, and overlap structure of downstream expert computation.

This tension becomes particularly pronounced for decomposition strategies that produce a large number of fine-grained matchings. While such schedules may appear attractive from a networking perspective, they can excessively fragment expert execution into small batches that suffer poor compute efficiency and limited overlap opportunities. Furthermore, because MoE communication matrices are often sparse and highly imbalanced, decomposition strategies may introduce substantial idle capacity and scheduling bubbles even while optimizing communication structure.

As a result, minimizing communication congestion alone is insufficient and can even become counterproductive. A decomposition that is communication optimal may expose too little computation per matching to sustain efficient pipelined execution. Conversely, coarser decompositions may preserve sufficiently large expert batches to better amortize compute overheads and improve overlap effectiveness despite incurring higher instantaneous communication contention.

Motivated by these observations, we instead view distributed MoE execution as a scheduling problem that jointly considers communication decomposition and compute efficiency. Rather than optimizing communication in isolation, the objective becomes balancing communication efficiency, overlap opportunities, and execution granularity to minimize end-to-end makespan.

3 Matrix Decompositions and Implications

A natural approach to executing MoE communication over reconfigurable optical fabrics is to decompose the all-to-all communication matrix into a sequence of matchings that can be scheduled over time. In this section, we discuss two decomposition strategies and their implications for end-to-end MoE execution.

3.1 Birkhoff–von Neumann Decomposition

Birkhoff–von Neumann (BvN) decomposition is a well-established technique in optical circuit switching for deriving communication schedules from a traffic matrix. Given a doubly stochastic matrix, BvN expresses it as a convex combination of permutation matrices: $A = \sum_i \lambda_i P_i$, where each permutation matrix P_i corresponds to a perfect matching. In the context of MoE execution, applying BvN to the all-to-all communication matrix directly produces a sequence of communication matchings between ranks.

However, BvN imposes a strong structural assumption: the input matrix must be doubly stochastic. In practice, MoE communication matrices are highly sparse, skewed, and imbalanced, making them far from bistochastic. Consequently, a preprocessing step such as Sinkhorn–Knopp normalization is required before decomposition can be applied.

Figure 2 illustrates this process. Starting from the original MoE communication matrix, Sinkhorn normalization redistributes communication mass across rows and columns to satisfy bistochastic constraints. While this enables BvN decomposition, it also alters the structure of the original communication demand and introduces artificial balancing into the schedule. The resulting decomposition often produces a large number of highly fragmented matchings, many of which carry only a small number of routed tokens.

From a networking perspective, these schedules are attractive because they reduce contention and produce perfectly incast- and outcast-free communication patterns. However, this same fragmentation creates significant execution challenges in MoE workloads.

Normalization introduces scheduling bubbles: Sinkhorn normalization redistributes communication mass to satisfy bistochastic constraints, introducing idle capacity that does not correspond to the original routing demand. As a result, some communicating pairs finish quickly while others continue transferring substantially larger token volumes, leaving communication bubbles within the schedule.

Fragmentation hurts compute efficiency: BvN often produces a large number of matchings, up to $O(n^2)$ in the worst case. While decomposition enables communication–compute overlap, excessively fragmented schedules expose only small token batches per matching. Figure 1 shows an approximately 250μs minimum execution overhead incurred by small batch sizes, while Figure 2 illustrates a real BvN decomposition from Mixtral-8x22B inference where several matchings carry only tens of routed tokens. Consequently, schedules that appear communication efficient can simultaneously exhibit poor end-to-end execution efficiency due to severely fragmented expert computation.

Small matchings amplify overlap inefficiencies: Small token batches produce short expert execution windows that are often insufficient to hide subsequent communication and reconfiguration overheads. For instance, a matching carrying only a few tens of routed tokens may complete expert computation too quickly to overlap the communication latency of the next matching, exposing both communication and reconfiguration costs.

■ ***Takeaway:*** *BvN highlights a fundamental tension in MoE execution: communication-optimal decompositions are not necessarily execution-optimal decompositions. In MoE workloads, decomposition simultaneously determines communication structure, compute granularity, and overlap opportunities, fundamentally coupling network scheduling with downstream execution efficiency.*

3.2 Greedy Max-Weight Decomposition

Motivated by the limitations of highly fragmented decompositions, we explore a simple greedy max-weight decomposition strategy in which each iteration computes a maximum-weight perfect matching using the Jonker-Volgenant algorithm [11]. Unlike BvN, this approach operates directly on the original MoE communication matrix without requiring bistochastic preprocessing or Sinkhorn normalization.

The algorithm repeatedly extracts the maximum-weight perfect matching from the residual matrix and subtracts the selected matching until all entries become zero. For an $n \times n$ matrix, each invocation of the Jonker-Volgenant algorithm computes a maximum-weight perfect matching in $O(n^3)$ time.

Figure 2 highlights the contrast between the two approaches. While BvN decomposes the normalized matrix into a large number of fragmented matchings carrying only small token volumes, the max-weight decomposition preserves large token counts within individual matchings and produces substantially fewer schedules overall.

Preserving larger token batches directly improves expert compute granularity and exposes longer expert execution windows capable of hiding subsequent communication and reconfiguration overheads. Furthermore, because the decomposition produces fewer matchings overall, it also reduces the number of synchronization points and topology reconfiguration events exposed during execution.

As a result, despite potentially introducing higher instantaneous communication contention, the overall execution often achieves significantly lower end-to-end makespan due to improved overlap behavior and substantially better compute efficiency.

■ ***Takeaway:*** *The key advantage of max-weight decomposition is not merely fewer matchings, but preserving large execution granularity that better aligns communication scheduling with expert compute behavior.*

3.3 Tradeoffs in Decomposition Granularity

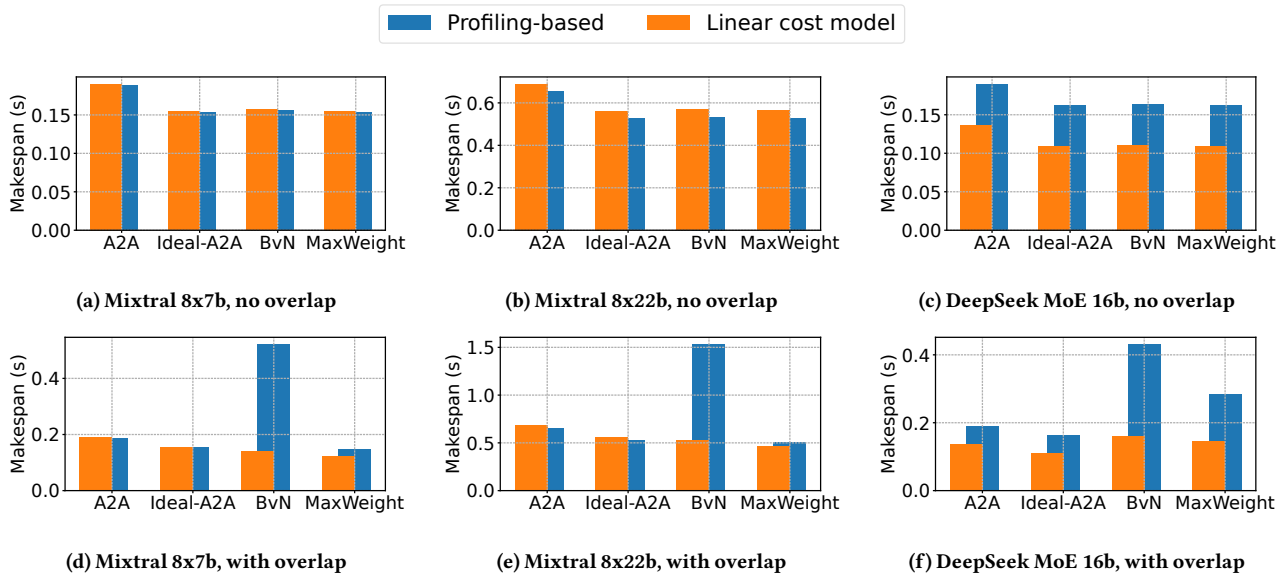
While the max-weight decomposition preserves larger token batches and reduces fragmentation, it also introduces a different tradeoff: individual matchings may themselves become highly imbalanced.

In particular, a single matching may contain communication pairs carrying vastly different token volumes. Since the completion time of a matching is determined by its most heavily loaded communication pair, smaller transfers may finish significantly earlier and remain idle while waiting for the bottleneck transfer to complete. Consequently, even though the decomposition produces fewer and denser matchings overall, imbalance within a matching can still introduce communication bubbles and underutilization.

This effect differs fundamentally from the fragmentation observed in BvN decomposition. Rather than producing many small matchings, the max-weight decomposition concentrates communication into a smaller number of dense matchings dominated by a subset of large transfers, as illustrated in Figure 2.

The decomposition additionally exposes a scheduling challenge regarding the ordering of matchings during execution. Large matchings expose long expert execution windows capable of hiding subsequent communication and reconfiguration overheads, while smaller residual matchings are more likely to leave the network exposed. Viewed from this perspective, the dispatch–compute–combine structure of MoE execution naturally resembles a three-machine flow-shop scheduling problem [18], where the objective is to minimize end-to-end makespan through the joint optimization of matching order and overlap behavior.

■ ***Takeaway:*** *Decomposition quality is not solely determined by the number of matchings produced. The internal structure, imbalance, and ordering of matchings fundamentally shape overlap opportunities and end-to-end execution efficiency.*



(a) Mixtral 8x7b, no overlap (b) Mixtral 8x22b, no overlap (c) DeepSeek MoE 16b, no overlap

(d) Mixtral 8x7b, with overlap (e) Mixtral 8x22b, with overlap (f) DeepSeek MoE 16b, with overlap

Figure 3: Makespan of the MoE forward pass under different decomposition strategies on the MMLU dataset, whose small prompt sizes often lead to small effective batch sizes during execution. Neither BvN nor greedy max-weight decompositions perform well, even under near-zero reconfiguration delay, compared to standard all-to-all communication over a static ring topology despite its severe congestion.

4 Evaluation

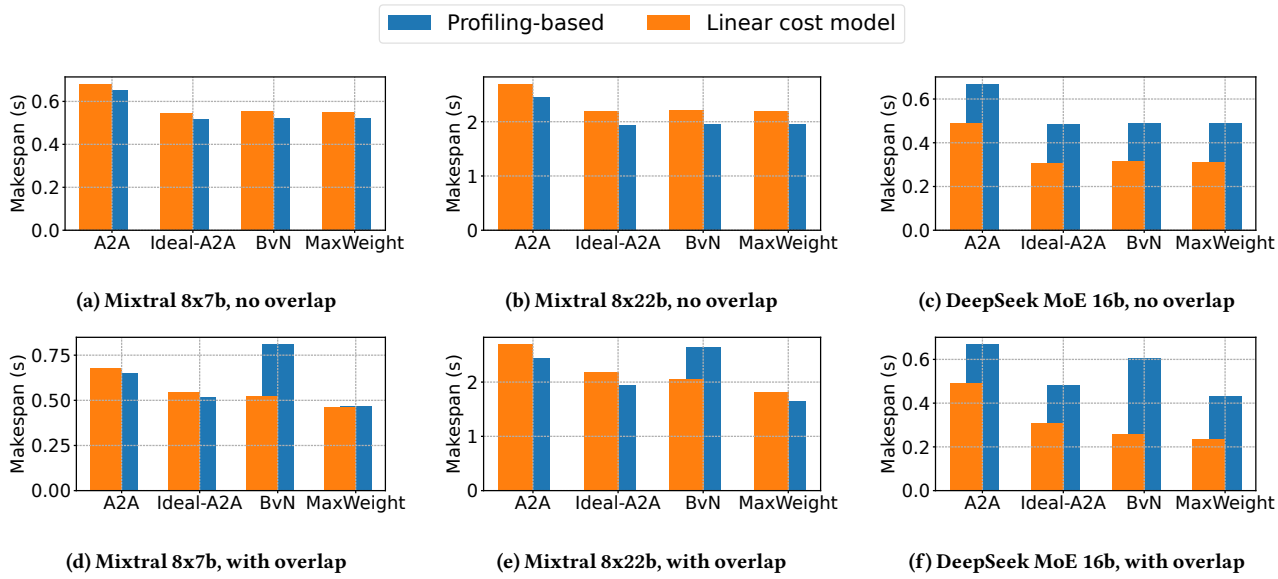
Our evaluation focuses on end-to-end MoE layer makespan and studies how decomposition granularity, overlap opportunities, and communication structure interact under different workload characteristics. To this end, we implement a trace-driven simulator using event queues to model the dispatch–compute–combine structure of the MoE forward pass, including all-to-all communication, expert execution, and communication–compute overlap. The simulator incorporates real routing traces and configurations from Mixtral 8×7B, Mixtral 8×22B, and DeepSeek MoE 16B models.

4.1 Setup

Comparisons: We compare sequential all-to-all execution, BvN decomposition, greedy max-weight decomposition, and an idealized congestion-free all-to-all baseline representing the theoretical completion time for a given communication matrix. We consider a system of 8 GPUs connected by a circuit-switched reconfigurable interconnect. The sequential all-to-all baseline operates over a static ring topology, while the decomposition-based strategies dynamically reconfigure circuits between communicating pairs after each matching. To isolate the impact of decomposition granularity and scheduling behavior, we deliberately assume a low reconfiguration delay of 10ns (e.g., Sirius [7]). Even under such optimistic assumptions, decomposition quality significantly impacts end-to-end makespan. In practice, larger reconfiguration delays can further influence the optimal scheduling strategy and the tradeoff between communication efficiency and execution granularity [9, 23, 28], opening an interesting direction for future work on decomposition-aware circuit scheduling under non-negligible reconfiguration costs.

Execution time: To model expert execution costs, we run experiments on RTX PRO 6000 GPUs and profile the execution time of MoE expert computation across different token batch sizes. We refer to this configuration as the *profiling-based* model in our evaluation. To additionally isolate the impact of decomposition granularity independent of hardware-specific effects, we also evaluate a synthetic linear compute cost model representing idealized compute scaling behavior. For static topologies, we use Gurobi [15] to solve for the optimal all-to-all completion time under link capacity constraints. For decomposition-based strategies, we analytically compute the completion time of each matching as the maximum communication time across all communicating pairs within the matching, i.e., the maximum transfer size divided by the available bandwidth. The decomposition-based strategies naturally enable communication–compute overlap. After the dispatch phase of the i-th matching completes, expert computation begins immediately while communication for the $(i+1)$-th matching progresses concurrently. In the presence of overlap, the event-driven simulator interleaves communication and expert execution events, allowing communication and reconfiguration overheads of subsequent matchings to be hidden behind ongoing expert computation whenever the exposed compute window is sufficiently large. In contrast, the sequential all-to-all baseline performs communication and computation strictly to completion without overlap.

Datasets: We evaluate these strategies across two workload regimes. The MMLU dataset consists primarily of small prompts, resulting in small effective token batches during execution. In contrast, the SPEED-BENCH throughput dataset contains large prompts ($\approx$ 2k tokens per prompt), producing substantially larger expert batches and greater opportunities for overlap and compute amortization.



(a) Mixtral 8x7b, no overlap

(b) Mixtral 8x22b, no overlap

(c) DeepSeek MoE 16b, no overlap

(d) Mixtral 8x7b, with overlap

(e) Mixtral 8x22b, with overlap

(f) DeepSeek MoE 16b, with overlap

Figure 4: Makespan of the MoE forward pass under different decomposition strategies on the SPEED-Bench throughput dataset, whose large prompt sizes (≈ 2k tokens per prompt) lead to substantially larger effective batch sizes during execution, thereby amortizing compute overheads. While BvN still suffers from the excessive number of matchings in its decomposition, greedy max-weight decomposition approaches the performance of an ideal congestion-free all-to-all and further benefits from communication–compute overlap.

4.2 Results

BvN severely underperforms under overlapped execution: Figure 3 shows the results for the MMLU dataset, whose small prompt sizes lead to small effective expert batches during execution. In this regime, the fragmentation introduced by BvN decomposition becomes particularly harmful. Our profiling of Mixtral 8×22B observed BvN producing up to 50 matchings, with many coefficients around 0.03, corresponding to only about 3% of routed tokens per matching. While decomposition theoretically enables communication–compute overlap (as seen in the linear compute cost model), these extremely small batches expose insufficient expert computation to hide subsequent communication and reconfiguration overheads. As a result, overlapped BvN execution performs significantly worse than its non-overlapped counterpart due to the accumulation of exposed scheduling bubbles and compute inefficiencies.

Static all-to-all can outperform decomposition under small batches with overlap: Figure 3 shows that under small-batch regimes, even a congestion-prone all-to-all over a static ring topology can outperform highly fragmented decomposition strategies. The small effective expert batches fail to amortize fixed compute overheads, causing decomposition-induced fragmentation to dominate the benefits of reduced communication contention.

Greedy max-weight decomposition benefits significantly from overlap: Figure 4 shows that under large-batch regimes, decomposition-based execution becomes substantially more effective. By preserving large token counts within each matching, the greedy max-weight decomposition exposes long expert execution windows that effectively hide subsequent communication (and can potentially hide reconfiguration overheads if they are non-negligible). As a result, the strategy significantly outperforms BvN decomposition and, in several settings, even approaches or surpasses the idealized congestion-free all-to-all baseline.

5 Conclusion

In this paper, we showed that communication-optimal decompositions are not necessarily execution-optimal for distributed MoE workloads over reconfigurable optical fabrics. Through trace-driven evaluation, we demonstrated that preserving large execution granularity is critical for efficient communication–compute overlap, allowing simple greedy max-weight decompositions to significantly outperform fragmented schedules such as BvN. Our results suggest that decomposition should be viewed not merely as a communication primitive, but as an execution scheduling mechanism, motivating future work on decomposition-aware scheduling under non-negligible circuit reconfiguration delays.

Acknowledgments

We would like to thank the anonymous reviewers for their insightful feedback. This research was supported by the NSF Future CoRe grant, award No. 2551372.

References

[1] Vamsi Addanki. When light bends to the collective will: A theory and vision for adaptive photonic scale-up domains. In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks*, HotNets '25, page 326–334, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3772356.3772395.

[2] Vamsi Addanki, Chen Avin, and Stefan Schmid. Mars: Near-optimal throughput with shallow buffers in reconfigurable datacenter networks. *Proc. ACM Meas. Anal. Comput. Syst.*, 7(1), mar 2023. doi:10.1145/3579312.

[3] Osayamen Jonathan Aimuyo, Byungsoo Oh, and Rachee Singh. FlashMoE: Fast Distributed MoE in a Single Kernel, 2025. URL: https://arxiv.org/abs/2506.04667.

[4] Daniel Amir, Nitika Saran, Tegan Wilson, Robert Kleinberg, Vishal Shrivastav, and Hakim Weatherspoon. Shale: A practical, scalable oblivious reconfigurable network. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 449–464, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3651890.3672248.

[5] Rukshani Athapathu and George Porter. Reconfigurability within collective communication algorithms. In *Proceedings of the 2nd Workshop on Networks for AI Computing*, NAIC '25, page 43–49, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3748273.3749203.

[6] Chen Avin and Stefan Schmid. Revolutionizing datacenter networks via reconfigurable topologies. *Commun. ACM*, 68(6):44–53, June 2025. doi:10.1145/3708980.

[7] Hitesh Ballani, Paolo Costa, Raphael Behrendt, Daniel Cletheroe, Istvan Haller, Krzysztof Jozwik, Fotini Karinou, Sophie Lange, Kai Shi, Benn Thomsen, and Hugh Williams. Sirius: A flat datacenter network with nanosecond optical switching. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, SIGCOMM '20, page 782–797, New York, NY, USA, 2020. Association for Computing Machinery. doi:10.1145/3387514.3406221.

[8] G BIRKHOFF. Tres observaciones sobre el algebra lineal. *Univ. Nac. Tucuman, Ser. A*, 5:147–154, 1946. URL: https://cir.nii.ac.jp/crid/1570572699525842816.

[9] Shaileshh Bojja Venkatakrishnan, Mohammad Alizadeh, and Pramod Viswanath. Costly circuits, submodular schedules and approximate carathéodory theorems. In *Proceedings of the 2016 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Science*, SIGMETRICS '16, page 75–88, New York, NY, USA, 2016. Association for Computing Machinery. doi:10.1145/2896377.2901479.

[10] Li-Wen Chang, Wenlei Bao, Qi Hou, Chengquan Jiang, Ningxin Zheng, Yinmin Zhong, Xuanrun Zhang, Zuquan Song, Chengji Yao, Ziheng Jiang, Haibin Lin, Xin Jin, and Xin Liu. FLUX: Fast Software-based Communication Overlap On GPUs Through Kernel Fusion, 2024. URL: https://arxiv.org/abs/2406.06858.

[11] David F. Crouse. On implementing 2d rectangular assignment algorithms. *IEEE Transactions on Aerospace and Electronic Systems*, 52(4):1679–1696, 2016. doi:10.1109/TAES.2016.140952.

[12] Nathan Farrington, George Porter, Sivasankar Radhakrishnan, Hamid Hajabdolali Bazzaz, Vikram Subramanya, Yeshaiahu Fainman, George Papen, and Amin Vahdat. Helios: a hybrid electrical/optical switch architecture for modular data centers. In *Proceedings of the ACM SIGCOMM 2010 Conference*, SIGCOMM '10, page 339–350, New York, NY, USA, 2010. Association for Computing Machinery. doi:10.1145/1851182.1851223.

[13] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.*, 23(1), 1 2022.

[14] Seokjin Go and Divya Mahajan. MoETuner: Optimized Mixture of Expert Serving with Balanced Expert Placement and Token Routing, 2025. URL: https://arxiv.org/abs/2502.06643.

[15] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2023. URL: https://www.gurobi.com.

[16] Xin Sunny Huang, Xiaoye Steven Sun, and T. S.Eugene Ng. Sunflow: Efficient optical circuit scheduling for coflows. *CoNEXT 2016 - Proceedings of the 12th International Conference on Emerging Networking EXperiments and Technologies*, pages 297–311, 12 2016. URL: https://dl.acm.org/doi/10.1145/2999572.2999592, doi:10.1145/2999572.2999592.

[17] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of Experts, 2024. URL: https://arxiv.org/abs/2401.04088.

[18] S M Johnson. Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68, 1954. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/nav.3800010110, doi:10.1002/nav.3800010110.

[19] Norman P. Jouppi and Andy Swing. A machine learning supercomputer with an optically reconfigurable interconnect and embeddings support. In *2023 IEEE Hot Chips 35 Symposium (HCS)*, pages 1–24, 2023. doi:10.1109/HCS59251.2023.10254691.

[20] Janardhan Kulkarni, Stefan Schmid, and Paweł Schmidt. Scheduling opportunistic links in two-tiered reconfigurable datacenters. In *Proceedings of the 33rd ACM Symposium on Parallelism in Algorithms and Architectures*, SPAA '21, page 318–327, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3409964.3461786.

[21] Abhishek Vijaya Kumar, Arjun Devraj, Darius Bunandar, and Rachee Singh. A case for server-scale photonic connectivity. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks*, HotNets '24, page 290–299, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3696348.3696856.

[22] Xudong Liao, Yijun Sun, Han Tian, Xinchen Wan, Yilun Jin, Zilong Wang, Zhenghang Ren, Xinyang Huang, Wenxue Li, Kin Fai Tse, Zhizhen Zhong, Guyue Liu, Ying Zhang, Xiaofeng Ye, Yiming Zhang, and Kai Chen. Mixnet: A runtime reconfigurable optical-electrical fabric for distributed mixture-of-experts training. In *Proceedings of the ACM SIGCOMM 2025 Conference*, SIGCOMM '25, page 554–574, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3718958.3750465.

[23] He Liu, Matthew K. Mukerjee, Conglong Li, Nicolas Feltman, George Papen, Stefan Savage, Srinivasan Seshan, Geoffrey M. Voelker, David G. Andersen, Michael Kaminsky, George Porter, and Alex C. Snoeren. Scheduling techniques for hybrid circuit/packet networks. In *Proceedings of the 11th ACM Conference on Emerging Networking Experiments and Technologies*, CoNEXT '15, New York, NY, USA, 2015. Association for Computing Machinery. doi:10.1145/2716281.2836126.

[24] William M. Mellette, Alex Forencich, Rukshani Athapathu, Alex C. Snoeren, George Papen, and George Porter. Realizing rotornet: Toward practical microsecond scale optical networking. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 392–414, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3651890.3672273.

[25] William M. Mellette, Rob McGuinness, Arjun Roy, Alex Forencich, George Papen, Alex C. Snoeren, and George Porter. Rotornet: A scalable, low-complexity, optical datacenter network. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, SIGCOMM '17, page 267–280, New York, NY, USA, 2017. Association for Computing Machinery. doi:10.1145/3098822.3098838.

[26] Matthew Nance Hall, Klaus-Tycho Foerster, Stefan Schmid, and Ramakrishnan Durairajan. A survey of reconfigurable optical networks. *Optical Switching and Networking*, 41:100621, 2021. URL: https://www.sciencedirect.com/science/article/pii/S1573427721000187, doi:10.1016/j.osn.2021.100621.

[27] George Porter, Richard Strong, Nathan Farrington, Alex Forencich, Pang Chen-Sun, Tajana Rosing, Yeshaiahu Fainman, George Papen, and Amin Vahdat. Integrating microsecond circuit switching into the data center. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM*, SIGCOMM '13, page 447–458, New York, NY, USA, 2013. Association for Computing Machinery. doi:10.1145/2486001.2486007.

[28] Mahir Rahman, Samuel Joseph, Nihar Kodkani, Behnaz Arzani, and Vamsi Addanki. Harvest: Adaptive photonic switching schedules for collective communication in scale-up domains, 2026. URL: https://arxiv.org/abs/2602.09188, arXiv:2602.09188.

[29] Sundararajan Renganathan and Nick McKeown. Chronos: Prescheduled circuit switching for llm training. In *Proceedings of the 2nd Workshop on Networks for AI Computing*, NAIC '25, page 89–97, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3748273.3749210.

[30] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer, 2017. URL: https://arxiv.org/abs/1701.06538.

[31] Luis Torrijos-Morán and Daniel Pérez-López. Industry insight: photonics to scale ai data centers. *npj Nanophotonics*, 3(1):8, 2026.

[32] Yen-Chieh Wu, Cheng-Shang Chang, Duan-Shin Lee, and H Jonathan Chao. Dynamic Hierarchical Birkhoff-von Neumann Decomposition for All-to-All GPU Communication, 2026. URL: https://arxiv.org/abs/2602.22756.

[33] Zhenguo Wu, Benjamin Klenk, Larry Dennison, and Keren Bergman. Actina: Adapting circuit-switching techniques for ai networking architectures. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '25, page 1211–1222, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3712285.3759842.

[34] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 Technical Report, 2025. URL: https://arxiv.org/abs/2505.09388.

[35] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, zhifeng Chen, Quoc V Le, and James Laudon. Mixture-of-Experts with Expert Choice Routing. In S Koyejo, S Mohamed, A Agarwal, D Belgrave, K Cho, and A Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 7103–7114. Curran Associates, Inc., 2022. URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/2f00ecd787b432c1d36f3de9800728eb-Paper-Conference.pdf.

[36] Yazhou Zu, Alireza Ghaffarkhah, Hoang-Vu Dang, Brian Towles, Steven Hand, Safeen Huda, Adekunle Bello, Alexander Kolbasov, Arash Rezaei, Dayou Du, Steve Lacy, Hang Wang, Aaron Wisner, Chris Lewis, and Henri Bahini. Resiliency at scale: Managing Google's TPUv4 machine learning supercomputer. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 761–774, Santa Clara, CA, April 2024. USENIX Association. URL: https://www.usenix.org/conference/nsdi24/presentation/zu.